



ML Lab 5

Name: Madhavi katekhaye

Section-Batch: B-B2

Roll No.: 22

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import (train_test_split, GridSearchCV, cross_
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score)
import warnings
warnings.filterwarnings("ignore")

sns.set_theme(style="whitegrid")
```

```
In [2]: df = pd.read_csv("diabetes.csv")
df.head()
```

```
Out[2]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction
0	6	148	72	35	0	33.6	0.625
1	1	85	66	29	0	26.6	0.351
2	8	183	64	0	0	23.3	0.672
3	1	89	66	23	94	28.1	0.167
4	0	137	40	35	168	43.1	2.278

```
In [3]: df.tail()
```

```
Out[3]:
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction
763	10	101	76	48	180	32.9	0.178
764	2	122	70	27	0	36.8	0.346
765	5	121	72	23	112	26.2	0.178
766	1	126	60	0	0	30.1	0.431
767	1	93	70	31	0	30.4	0.336

```
In [4]: df.shape
```

Out[4]: (768, 9)

```
In [5]: df.isnull().sum()
```

Out[5]: Pregnancies 0
Glucose 0
BloodPressure 0
SkinThickness 0
Insulin 0
BMI 0
DiabetesPedigreeFunction 0
Age 0
Outcome 0
dtype: int64

```
In [6]: df.info()
```

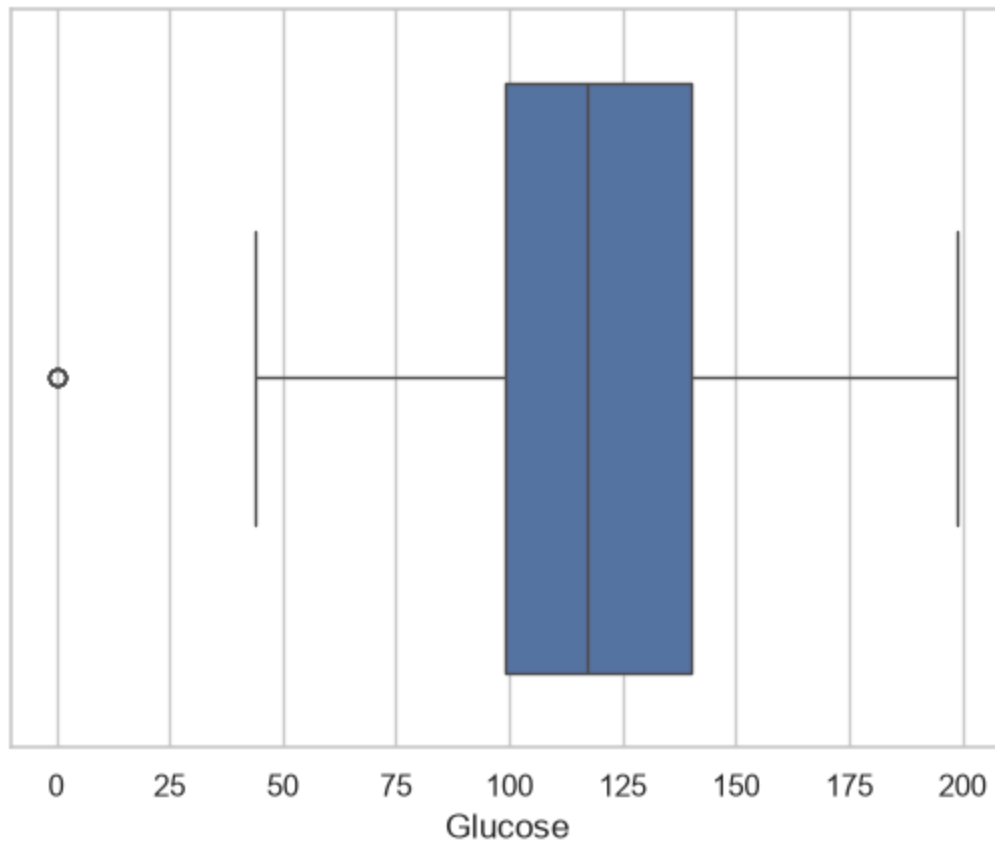
<class 'pandas.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
Column Non-Null Count Dtype
--- -
0 Pregnancies 768 non-null int64
1 Glucose 768 non-null int64
2 BloodPressure 768 non-null int64
3 SkinThickness 768 non-null int64
4 Insulin 768 non-null int64
5 BMI 768 non-null float64
6 DiabetesPedigreeFunction 768 non-null float64
7 Age 768 non-null int64
8 Outcome 768 non-null int64
dtypes: float64(2), int64(7)
memory usage: 54.1 KB

```
In [7]: df.describe()
```

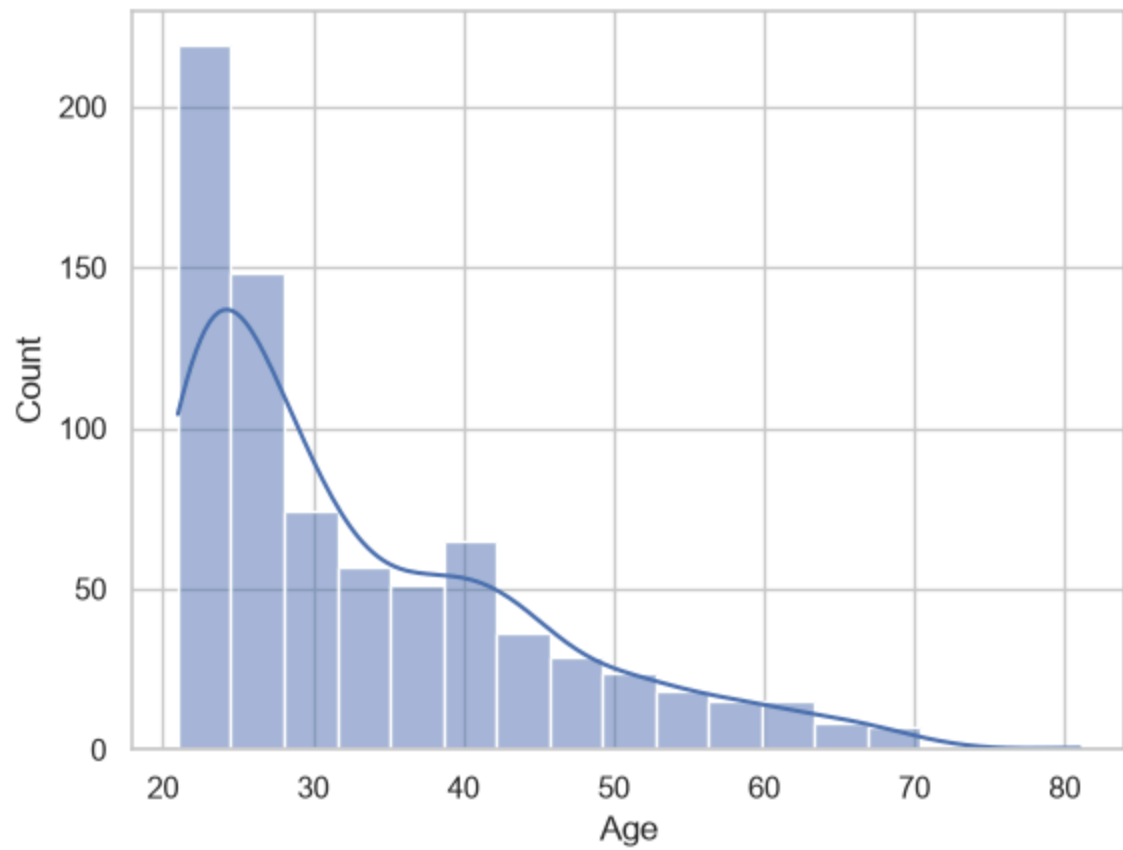
Out[7]:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	Diabe
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	
mean	3.845052	120.894531	69.105469	20.536458	79.799479	31.992578	
std	3.369578	31.972618	19.355807	15.952218	115.244002	7.884160	
min	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	1.000000	99.000000	62.000000	0.000000	0.000000	27.300000	
50%	3.000000	117.000000	72.000000	23.000000	30.500000	32.000000	
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	

```
In [8]: sns.boxplot(x=df["Glucose"])  
plt.show()
```



```
In [9]: sns.histplot(df["Age"], kde=True)  
plt.show()
```



```
In [10... features = df.drop(columns="Outcome").columns

df[features].hist(
    figsize=(14, 12),
    bins=20,
    kde=True
)

plt.suptitle(
    "Distribution of Diabetes Dataset Features",
    fontsize=16
)

plt.tight_layout()
plt.show()
```

AttributeError Traceback (most recent call last)

Cell In[10], line 3

```
1 features = df.drop(columns="Outcome").columns
2
----> 3 df[features].hist(
4     figsize=(14, 12),
5     bins=20,
6     kde=True
```

File ~\AppData\Roaming\Python\Python314\site-packages\pandas\plotting_core.py:258, in hist_frame(data, column, by, grid, xlabelsize, xrot, ylabelsize, yrot, ax, sharex, sharey, figsize, layout, bins, backend, legend, **kwargs)

```
168 """
169 Make a histogram of the DataFrame's columns.
170
171 (...) 255 >>> hist = df.hist(bins=3)
256 """
257 plot_backend = get_plot_backend(backend)
--> 258 return plot_backend.hist_frame(
259     data,
260     column=column,
261     by=by,
262     grid=grid,
263     xlabelsize=xlabelsize,
264     xrot=xrot,
265     ylabelsize=ylabelsize,
266     yrot=yrot,
267     ax=ax,
268     sharex=sharex,
269     sharey=sharey,
270     figsize=figsize,
271     layout=layout,
272     legend=legend,
273     bins=bins,
274     **kwargs,
275 )
```

File ~\AppData\Roaming\Python\Python314\site-packages\pandas\plotting_matplotlib\hist.py:563, in hist_frame(data, column, by, grid, xlabelsize, xrot, ylabelsize, yrot, ax, sharex, sharey, figsize, layout, bins, legend, **kwargs)

```
561 if legend and can_set_label:
562     kwds["label"] = col
--> 563 ax.hist(data[col].dropna().values, bins=bins, **kwds)
564 ax.set_title(col)
565 ax.grid(grid)
```

[... skipping hidden 2 frame]

File ~\AppData\Roaming\Python\Python314\site-packages\matplotlib\axes_axes.py:7676, in Axes.hist(self, x, bins, range, density, weights, cumulative, bottom, histtype, align, orientation, rwidth, log, color, label, stacked,

```

**kwargs)
7674 p = patch[0]
7675 kwargs.update(next(style_gen))
-> 7676 p._internal_update(kwargs)
7677 if lbl is not None:
7678     p.set_label(lbl)

```

```

File ~\AppData\Roaming\Python\Python314\site-packages\matplotlib\artist.p
y:1313, in Artist._internal_update(self, kwargs)
1306 def _internal_update(self, kwargs):
1307     """
1308     Update artist properties without prenormalizing them, but gener
ating
1309     errors as if calling `set`.
1310
1311     The lack of prenormalization is to maintain backcompatibility.
1312     """
-> 1313     return self.update_props(
1314         kwargs, "{cls.__name__}.set() got an unexpected keyword arg
ument "
1315         "{prop_name!r}")

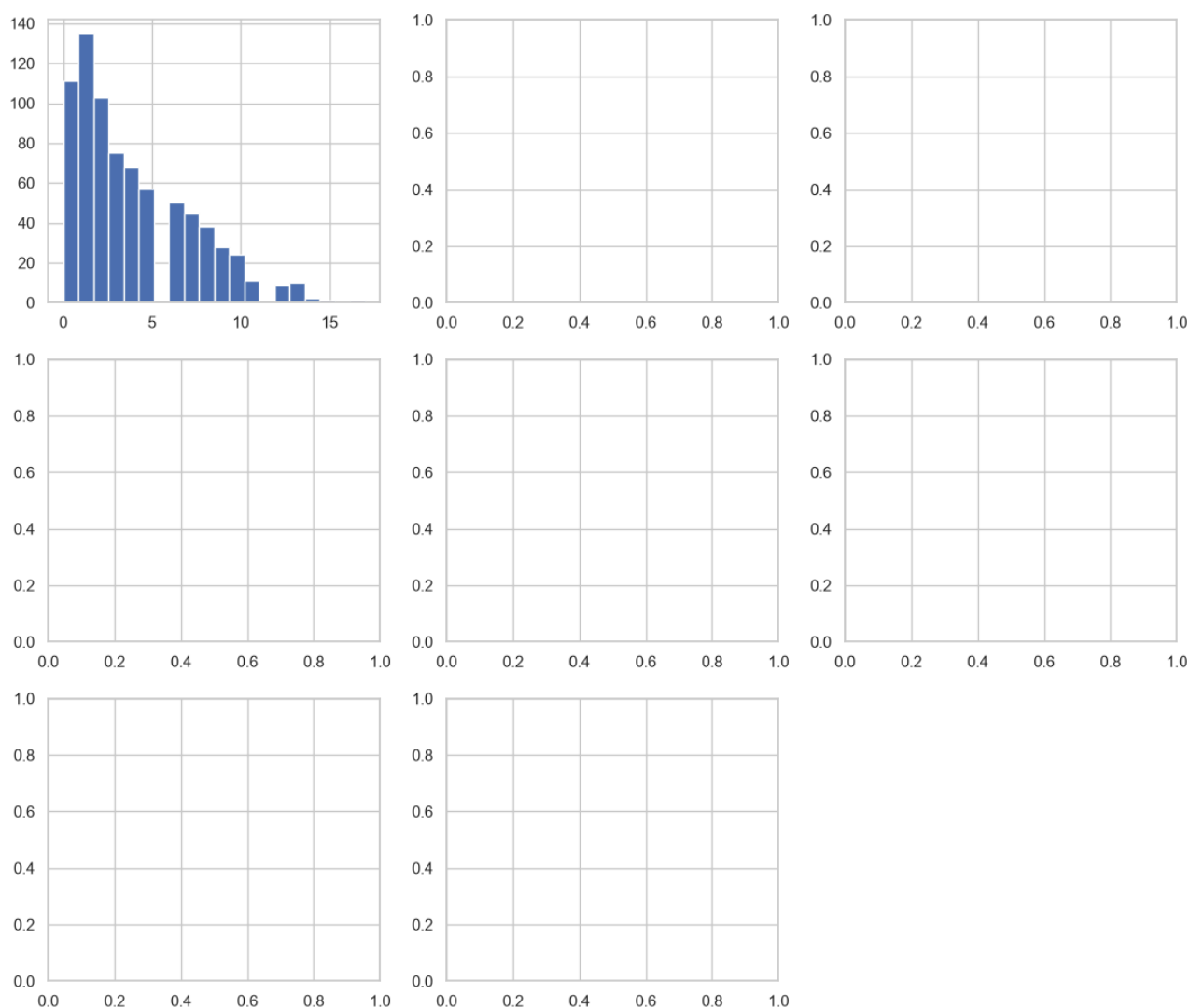
```

```

File ~\AppData\Roaming\Python\Python314\site-packages\matplotlib\artist.p
y:1271, in Artist._update_props(self, props, errfmt)
1269     func = getattr(self, f"set_{k}", None)
1270     if not callable(func):
-> 1271         raise AttributeError(
1272             errfmt.format(cls=type(self), prop_name=k),
1273             name=k)
1274     ret.append(func(v))
1275 if ret:

```

AttributeError: Rectangle.set() got an unexpected keyword argument 'kde'



```
In [ ]: zero_columns = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
zero_counts = (df[zero_columns]==0).sum()
print("Zero Values")
print(zero_counts)
```

```
Zero Values
Glucose      5
BloodPressure 35
SkinThickness 227
Insulin      374
BMI          11
dtype: int64
```

```
In [ ]: df_clean = df.copy()
df_clean[zero_columns]=df_clean[zero_columns].replace(0,np.nan)
print("Missing Values after replacing invalid zeros:")
print(df_clean.isnull().sum())
```

Missing Values after replacing invalid zeros:

Pregnancies	0
Glucose	5
BloodPressure	35
SkinThickness	227
Insulin	374
BMI	11
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

```
In [ ]: for col in zero_columns:
        df_clean[col]=df_clean[col].fillna(df_clean[col].median())
        print("Missing Values after median:")
        print(df_clean.isnull().sum())
```

Missing Values after median:

Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

```
In [ ]: X = df_clean.drop("Outcome",axis=1)

y= df_clean["Outcome"]

print("X shape",X.shape)
print(" y Shape ",y.shape)
```

X shape (768, 8)
y Shape (768,)

```
In [ ]: X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.20,random_state=42)
        print("Training samples:",X_train.shape[0])
        print("testing samples:",X_test.shape[0])
```

Training samples: 614
testing samples: 154

```
In [ ]: print("Training distribution:")
        print(y_train.value_counts(normalize=True))

        print("Testing distribution:")
        print(y_test.value_counts(normalize=True))
```



```

Training distribution:
Outcome
0    0.651466
1    0.348534
Name: proportion, dtype: float64
Testing distribution:
Outcome
0    0.649351
1    0.350649
Name: proportion, dtype: float64

```

```

In [ ]: scaler= StandardScaler()
X_train_scaled = scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

```

```

In [ ]: k=5

```

```

In [ ]: knn =KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train_scaled,y_train)

```

```

Out[ ]: ▼ KNeighborsClassifier
KNeighborsClassifier()

```

```

In [ ]: y_pred = knn.predict(X_test_scaled)
print("Prediction")
print(y_pred)

```

```

Prediction
[1 0 0 1 0 0 0 1 0 1 0 1 0 0 0 1 1 0 1 0 0 1 0 1 0 0 1 0 1 0 0 0 0 1 1 0
0
0 1 0 1 0 0 0 0 0 0 0 0 1 0 0 1 0 0 1 0 1 0 1 0 0 1 0 0 1 0 0 1 0 0 0
1
0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 1 1 1 0 0 0 0 0 1 0 1 0 1 1
1
1 1 0 0 0 0 0 1 0 1 0 0 1 0 1 1 0 0 0 0 0 0 1 1 0 0 0 0 1 0 1 0 0 0 0 1
0
0 0 0 0 1 0]

```

```

In [ ]: y_prob = knn.predict_proba(X_test_scaled)[: ,1]
print(y_prob)

```

```

[0.6 0.2 0.2 0.6 0.  0.2 0.2 0.8 0.  0.6 0.4 0.6 0.  0.  0.  0.6 0.8 0.
1.  0.2 0.4 1.  0.4 0.8 0.2 0.  0.6 0.  0.8 0.  0.  0.  0.4 0.6 0.8 0.2
0.  0.  0.6 0.4 0.6 0.4 0.  0.2 0.2 0.4 0.  0.  0.6 0.2 0.2 0.8 0.2 0.
0.6 0.2 0.6 0.  0.8 0.  0.8 0.4 0.2 1.  0.  0.4 1.  0.  0.4 0.8 0.2 0.
0.2 0.6 0.  0.4 0.  0.8 0.  0.  0.  0.  0.2 0.2 0.  0.4 0.2 0.  0.4
0.6 0.8 0.2 0.  0.  0.4 0.6 0.6 0.8 0.  0.2 0.  0.  0.2 0.8 0.  0.8 0.
0.8 0.6 0.6 0.8 0.6 0.2 0.4 0.4 0.2 0.  1.  0.  0.6 0.  0.2 0.6 0.2 0.8
0.6 0.4 0.  0.2 0.  0.4 0.2 0.8 0.6 0.  0.  0.  0.  0.8 0.  0.6 0.  0.
0.2 0.  0.6 0.4 0.4 0.  0.2 0.  0.8 0. ]

```

```
In [ ]: accuracy = accuracy_score(y_test,y_pred)
print(f"Accuracy:{accuracy:.4f}")
```

Accuracy:0.7532

```
In [ ]: precision = precision_score(y_test,y_pred)
print(f"Precision:{precision:.4f}")
```

Precision:0.6600

```
In [ ]: recall = recall_score(y_test,y_pred)
print(f"Recall: {recall:.4f}")
```

Recall: 0.6111

```
In [ ]: f1=f1_score(y_test,y_pred)
print(f"F1 score :{f1:.4f}")
```

F1 score :0.6346

```
In [ ]: print(classification_report(y_test, y_pred, target_names=["NoDiabetic", "
```

	precision	recall	f1-score	support
NoDiabetic	0.80	0.83	0.81	100
Diabetic	0.66	0.61	0.63	54
accuracy			0.75	154
macro avg	0.73	0.72	0.72	154
weighted avg	0.75	0.75	0.75	154

```
In [ ]: cm=confusion_matrix(y_test,y_pred)
print(cm)
```

```
[[83 17]
 [21 33]]
```

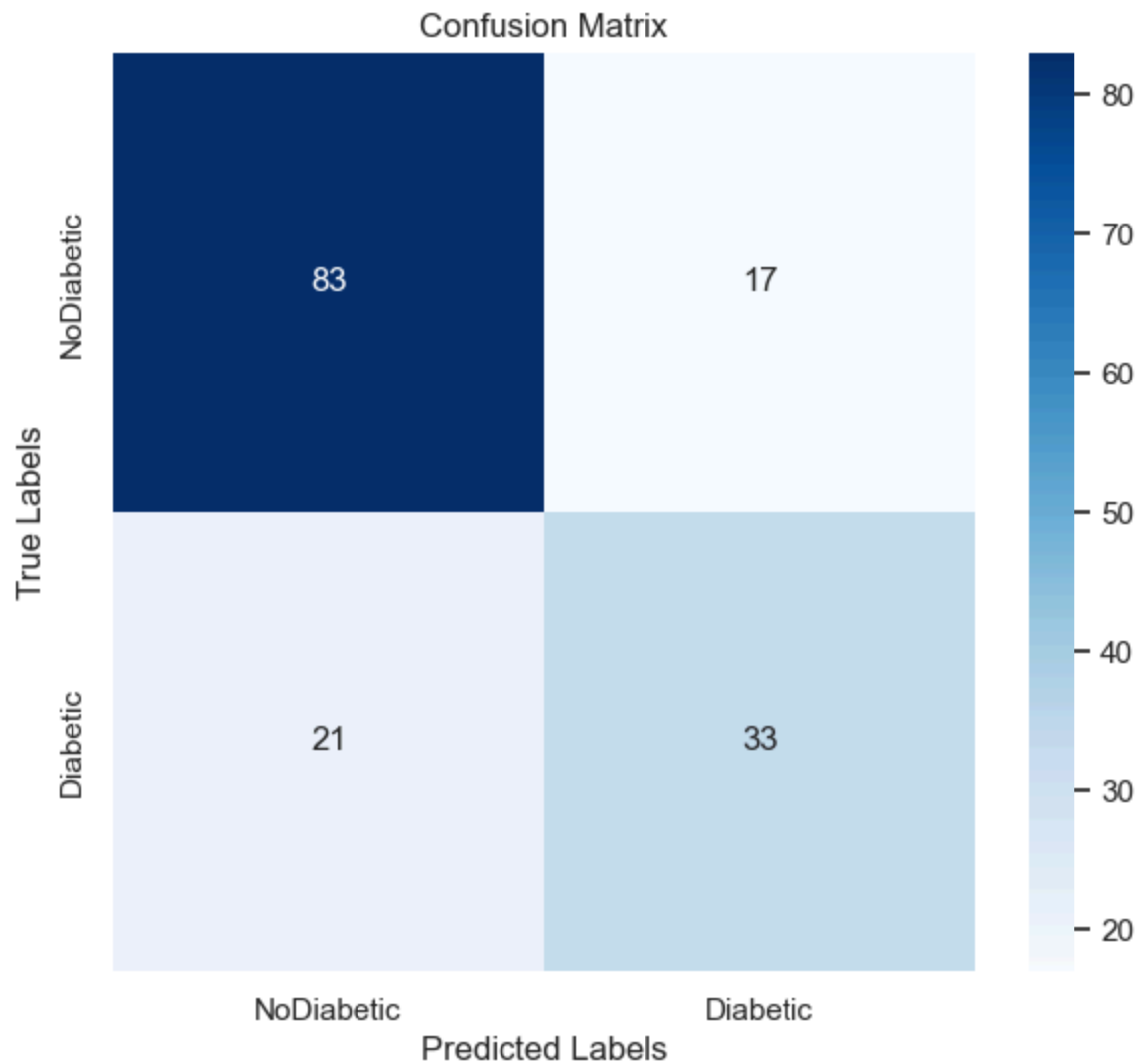
```
In [ ]: plt.figure(figsize=(7, 6))

class_names = ["NoDiabetic", "Diabetic"]

sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=class_name

plt.title("Confusion Matrix")
plt.xlabel("Predicted Labels")
plt.ylabel("True Labels")

plt.show()
```



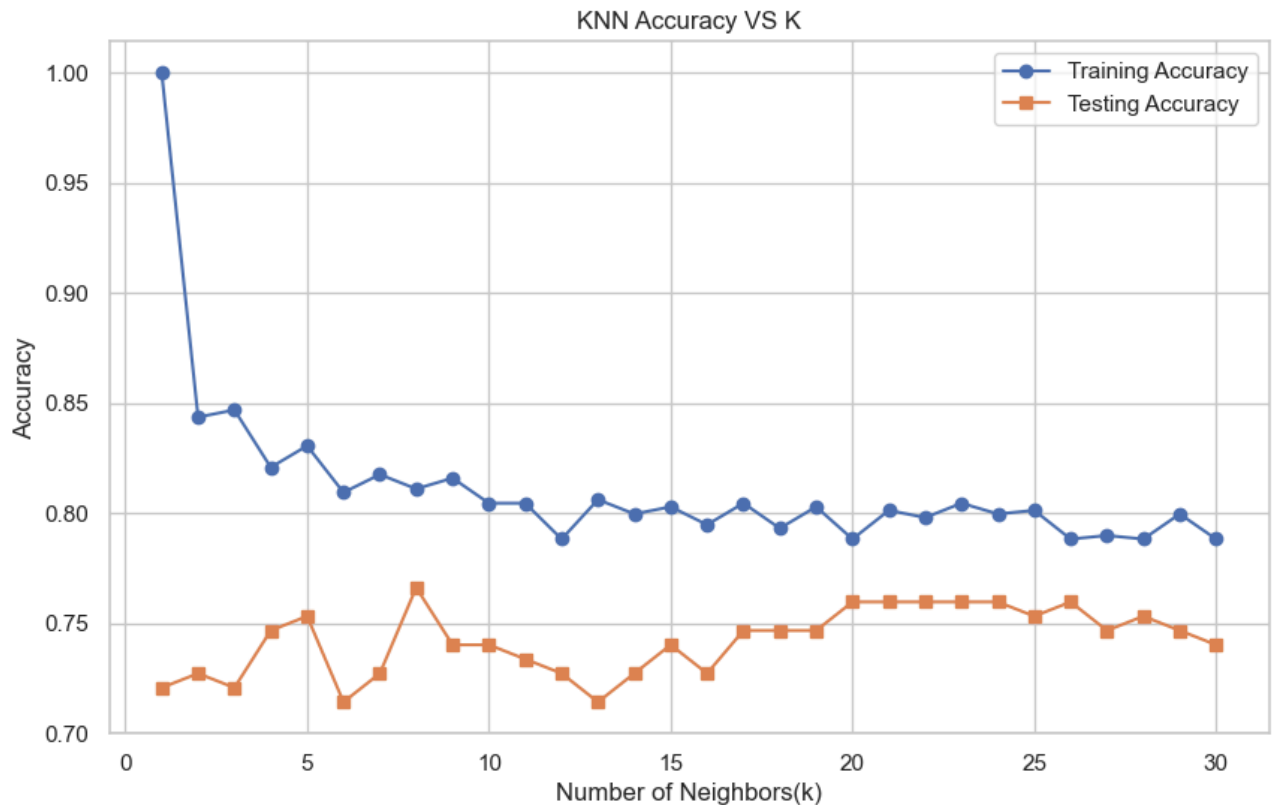
```
In [ ]: k_values = range(1,31)
train_accuracy = []
test_accuracy = []

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train_scaled,y_train)
    train_accuracy.append(model.score(X_train_scaled,y_train))
    test_accuracy.append(model.score(X_test_scaled,y_test))
```

```
In [ ]: plt.figure(figsize=(10,6))
plt.plot(k_values,train_accuracy,marker="o",label="Training Accuracy")
plt.plot(k_values,test_accuracy,marker="s",label="Testing Accuracy")

plt.xlabel("Number of Neighbors(k)")
plt.ylabel("Accuracy")
plt.title("KNN Accuracy VS K")
plt.legend()
plt.grid(True)
```

```
plt.show()
```



```
In [ ]: best_k_accuracy = k_values[np.argmax(test_accuracy)]  
best_accuracy = max(test_accuracy)
```

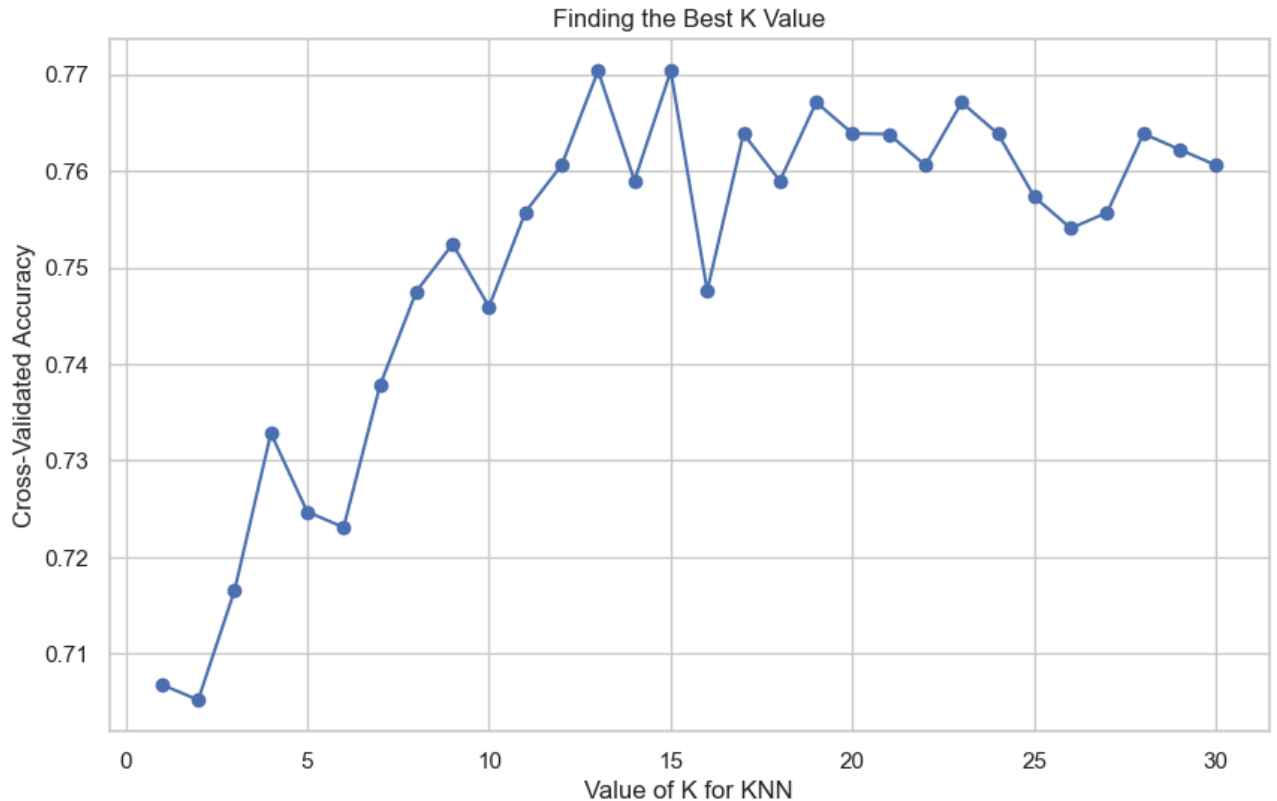
```
print("Best K",best_k_accuracy)  
print("Best Test Accuracy:",best_accuracy)
```

```
Best K 8  
Best Test Accuracy: 0.7662337662337663
```

```
In [ ]: from sklearn.model_selection import StratifiedKFold, cross_val_score  
from sklearn.neighbors import KNeighborsClassifier  
  
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)  
cv_scores = []  
  
for k in range(1, 31):  
    model = KNeighborsClassifier(n_neighbors=k)  
    scores = cross_val_score(model, X_train_scaled, y_train, cv=cv, scoring='accuracy')  
    cv_scores.append(scores.mean())
```

```
In [ ]: plt.figure(figsize=(10, 6))  
plt.plot(range(1, 31), cv_scores, marker="o")  
plt.xlabel("Value of K for KNN")  
plt.ylabel("Cross-Validated Accuracy")  
plt.title("Finding the Best K Value")  
plt.grid(True)
```

```
plt.show()
```



```
In [ ]: best_k_cv=np.argmax(cv_scores)+1
print("Best K Based on Cross Validation:",best_k_cv)
print("Best CV Accuracy:",max(cv_scores))
```

Best K Based on Cross Validation: 13

Best CV Accuracy: 0.7703718512594963

```
In [ ]: param_grid={"n_neighbors":list(range(3,31,2)), "weights":["uniform","dista",
    "metric":["eucliden","manhattan","minkowski"],
    "p":[1,2]
}
```

```
In [ ]: from sklearn.model_selection import GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
```

```
grid_search = GridSearchCV(
    estimator=KNeighborsClassifier(),
    param_grid=param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid_search.fit(X_train_scaled, y_train)
print("Grid Search completed.")
```

Grid Search completed.

```
In [ ]: print("Best Parameters:")  
        print(grid_search.best_params_)
```

```
Best Parameters:  
{'metric': 'minkowski', 'n_neighbors': 19, 'p': 2, 'weights': 'uniform'}
```

```
In [ ]: print("Best Cross- Validation Accuracy",grid_search.best_score_)
```

```
Best Cross- Validation Accuracy 0.7769025723044115
```

```
In [ ]:
```